

Voice Orchestrator — Complete API Reference

All mobile-facing endpoints: identity, enrollments, devices & search, favorites, scene mappings, automations, boards, voice enable, VAPI.

Version 1.3 | August 27, 2026 | Tetradapt | everything verified in production

What changed in 1.3. Multi-home support is live: login/GET /me return ALL of the user's homes; default_home_id is now guaranteed STABLE (the first-registered home — adding a home never flips the default). New §1.1 defines the home-switcher contract the app should implement. **1.2.** §6's action matrix now covers EVERY domain, verified against the home's own HA service registry: toggle/turn_on/turn_off work for lights, switches, fans, covers, media players, remotes, scripts, climate and input_booleans — not just automations — plus a copy-paste test recipe. **1.1:** new §6 — GET /automations lists every HA automation in a home (the automations pull for the mobile app), and POST /automations/trigger takes an optional action (toggle / turn_on / turn_off) so on/off devices can actually be turned OFF — the boards-page “can't turn the lamp off” bug. POST /favorites/{id}/fire accepts the same override. New §7 indexes the boards endpoints (deep dive: boards_api_guide.pdf).

Basics

Item	Value
Base URL (\$BASE)	https://voiceorchestrator.homeadapt.us/api/v1/voice-auth
Auth	Authorization: Bearer <login token (JWT) or static platform key>
Login token	from POST /auth/login; 30-day expiry; identifies the USER; sending a foreign user_ref or home_id → 403
Static keys	sk_ios_/sk_and_/sk_web_ — identify only the platform; legacy, still work during the transition
Error envelope	{"error": "<message>", "code": "<CODE>"} on every non-2xx

Session model: login once with email+password → get a signed token → send it as the bearer on every call → any 401 means re-login. Logout = delete the stored token. Passwords are never stored on the device or re-sent.

Endpoint index

Group	Endpoints
Identity	POST /auth/signup · POST /auth/login · GET /me · POST /auth/change-password
Enrollments (voice-gate)	POST/GET /enrollments · GET/DELETE /enrollments/{id} · PATCH /enrollments/{id}/status · GET /check · GET /voice-gated · GET /challenges · GET /challenges/{id}
Devices & search	GET /devices/discover · GET /items/search · GET /automations/discover
Favorites	POST/GET /favorites · DELETE /favorites/{ref} · PATCH /favorites/reorder · POST /favorites/{id}/fire
Scene mappings	POST/GET /scene-mappings · GET/PATCH/DELETE /scene-mappings/{id}
Automations	GET /automations · POST /automations/trigger

Group	Endpoints
Boards (dashboards)	GET /dashboards · GET /dashboards/config
Voice enable (VAPI)	POST/GET/DELETE /voice-enable
Phone mappings (server/VAPI internal)	POST/GET /phone-mappings · DELETE /phone-mappings/{id} · GET /phone-lookup

1 · Identity

POST /auth/signup — create account (pending approval)

```
curl -s -X POST "$BASE/auth/signup" -H "Content-Type: application/json" \
  -d '{"email":"new@example.com","password":"min-8-chars","full_name":"New User"}'
```

201 pending_approval (admin activates + attaches home before login works) · 409 EMAIL_EXISTS · 400 VALIDATION · 429 RATE_LIMITED (5/15 min/IP). Login before activation → 403 PENDING_APPROVAL.

POST /auth/login — get token + identity in one call

```
curl -s -X POST "$BASE/auth/login" -H "Content-Type: application/json" \
  -d '{"email":"scott@example.com","password":"*****"}'
```

```
200: { "token": "eyJ...", "token_type": "Bearer", "expires_in": 2592000,
      "user_ref": "scott_mobile", "user_id": "scott_mobile",
      "username": "scottmeyers", "email": "smeyersne@gmail.com",
      "full_name": "scott meyers",
      "homes": [ { "home_id": "scott_home", "name": "Scott's House" },
                  { "home_id": "ne_qli_1", "name": "NE QLI Office" } ],
      "default_home_id": "scott_home" }
```

Cache user_ref + default_home_id — use them everywhere the app previously hardcoded scott_mobile/scott_home. user_id + email are for feedback reports. 401 wrong credentials · 403 PENDING_APPROVAL · 429 after 5 failures (15-min lock).

GET /me — refresh identity on startup

```
curl -s "$BASE/me" -H "Authorization: Bearer $TOKEN"
```

Same payload minus token fields. 401 → token expired → login screen. Requires a login token (static keys are rejected — they carry no identity).

1.1 · Multiple homes — the home-switcher contract

A user can own SEVERAL homes (e.g. house + office). There is no server-side “current home”: the login token identifies the USER, and every data endpoint takes an explicit home_id. Switching homes is purely client state.

Rule	Contract
Source of truth	homes[] from POST /auth/login or GET /me — never hardcode home_id
Default	default_home_id = the user's FIRST-registered home; stable — adding homes never changes it

Rule	Contract
Selected home	keep a persisted selectedHomelId, initialized to default_home_id; “switching” = change it and refetch — no re-login, no new token
On every launch	refresh /me; if stored selectedHomelId is no longer in homes[] fall back to default_home_id; homes empty → “awaiting home setup” screen
Picker UI	show a switcher only when homes.length > 1 (single-home users see no change); list by name, checkmark the selected one
Per-home data	favorites, boards/dashboards, device search, automations, voice-auth enrollments and VAPI numbers are ALL keyed by (user_ref, home_id) — each home has its own; expect an empty favorites list on a freshly added home
Wrong home_id	a home_id the user doesn't own → 403 — treat as “re-sync /me”, not as an auth failure

Concretely: on switch, cancel in-flight requests, then refetch GET /favorites?home_id=..., GET /dashboards/config?... and any cached /devices/discover data for the new home. A voice-gated favorite in home B needs its own enrollment under home B.

POST /auth/change-password

```
curl -s -X POST "$BASE/auth/change-password" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"current_password":"old-one","new_password":"new-min-8-chars"}'
```

204 changed (token stays valid) · 403 wrong current password · 400 policy · 429 lockout. Forgotten password: admin reset only (for now).

2 · Voice-auth enrollments (gate an automation behind a spoken phrase)

POST /enrollments — enable “Voice Protect” on an automation

```
curl -s -X POST "$BASE/enrollments" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{
    "user_ref": "scott_mobile", "home_id": "scott_home",
    "automation_name": "Main Lights On",
    "ha_service": "script", "ha_entity": "main_lights_on" }'
```

ha_service ∈ scene/script/switch/light/lock/cover/media_player/climate/input_boolean/fan; ha_entity is the suffix only (main_lights_on, not script.main_lights_on). 201 with id, status ACTIVE, max_attempts 3, cooldown 30s. Idempotent on (user_ref, automation_id). The gate matches on the entity SUFFIX across domains — an enrollment on main_lights_on gates script.main_lights_on and automation.main_lights_on alike.

The rest of the group

Call	What it does
GET /enrollments?user_ref=...[&status=...]	list a user's enrollments → {count, items}
GET /enrollments/{id}	one enrollment's detail

Call	What it does
PATCH /enrollments/{id}/status { "status": "PAUSED" "ACTIVE" }	pause / resume
DELETE /enrollments/{id}	revoke permanently → 204
GET /check?user_ref=...&automation_id=...	pre-flight before a VAPI session → exists, status, attempts_remaining, cooldown_remaining_seconds
GET /voice-gated?user_ref=...[&home_id=...]	read-only list of every entity that would answer 409 for this user
GET /challenges?user_ref=...&limit=10	audit log of attempts (PASS/FAIL/TIMEOUT + vapi_call_id)
GET /challenges/{log_id}	one attempt's detail

3 · Devices & search

GET /devices/discover?home_id=... — physical devices

One row per HA device: device_id, name, manufacturer, model, area, primary_entity_id (the controllable surface), is_controllable, all_entities. Cached 60s server-side. Sensor-only devices have is_controllable:false and cannot be favorited.

GET /items/search — the “Add favorite” picker (devices + scenes + scripts + automations)

```
curl -s -G "$BASE/items/search" -H "Authorization: Bearer $TOKEN" \
  --data-urlencode "home_id=$HOME" --data-urlencode "q=bat" \
  --data-urlencode "user_ref=$USER"
```

Param	Req	Notes
home_id	yes	which home to search
q	no	case-insensitive substring on name + entity_id + device_id
kind	no	comma-filter: device,scene,script,automation,entity
user_ref	no	populates is_favorited + favorite_id per row
limit	no	default 200, max 500

GET /automations/discover?home_id=... — voice-ELIGIBLE candidates for the Voice-Protect enrollment picker (scenes/scripts/devices). Despite the name it does NOT return automation.* entities — for those use GET /automations (§6).

4 · Favorites (home-screen tiles)

POST /favorites — add by device OR entity

```
# by device (server resolves entity, domain, friendly_name from HA):
curl -s -X POST "$BASE/favorites" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"user_ref": "$USER", "home_id": "$HOME", "device_id": "6b86cd8c..."}'
```

```
# by entity (scenes/scripts/automations; friendly_name recommended):
-d '{"user_ref":"$USER","home_id":"$HOME",
  "entity_id":"scene.good_morning","friendly_name":"Good Morning"}'
```

Exactly ONE of device_id / entity_id (both or neither → 400). 201 returns the favorite incl. its id (UUID — store it). Duplicates → 400. Sensor-only device → 400 NO_CONTROLLABLE_ENTITY. **Locks auto-enroll:** lock.* favorites return voice_auth_required:true + voice_auth_enrollment_id — firing them always requires the voice challenge.

GET /favorites?user_ref=...&home_id=... — ordered list; each row carries voice_auth_required for the lock badge.

DELETE /favorites/{ref} — two forms

```
# by favorite id (from create response / list):
curl -s -X DELETE -H "Authorization: Bearer $TOKEN" "$BASE/favorites/<uuid>" # 204

# ONE-STEP by device_id or entity_id (no lookup):
curl -s -X DELETE -H "Authorization: Bearer $TOKEN" \
  "$BASE/favorites/6b86cd8c...?home_id=$HOME" # 204
# user_ref inferred from token; static-key callers add &user_ref=$USER
```

PATCH /favorites/reorder — drag-and-drop

```
curl -s -X PATCH "$BASE/favorites/reorder" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"items":[{"id":"<id-1>","position":0}, {"id":"<id-2>","position":1}]}'
```

POST /favorites/{id}/fire — one-tap activation

Server picks the HA action by domain: scene/script/light/switch → turn_on; automation → trigger; lock → unlock. Body (optional): {"action":"toggle"|"turn_on"|"turn_off"|...} overrides the default — same contract as /automations/trigger (§6). 200 {success, latency_ms}. **409 ENROLLMENT_REQUIRED** when voice-gated (always for locks) — launch the VAPI session with the returned automation_id + home_id instead of retrying.

5 · Scene mappings (spoken name → HA webhook)

```
curl -s -X POST "$BASE/scene-mappings" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{
  "home_id": "scott_home", "scene_name": "Movie Night",
  "webhook_id": "movie_night_1751404299018" }'
```

Call	What it does
POST /scene-mappings	create (names normalized to lowercase) → 201
GET /scene-mappings?home_id=...	list a home's scenes
GET /scene-mappings/{id}	detail
PATCH /scene-mappings/{id}	update scene_name / webhook_id / is_active (all optional)

Call	What it does
DELETE /scene-mappings/{id}	remove → 204

6 · Automations — list, run, toggle (NEW in 1.1)

GET /automations — every HA automation in a home

The automations pull for the mobile app: one row per automation.* entity, sorted by name, read from the home's Home Assistant with the orchestrator's stored token (the app never holds HA credentials). Served through a ~10s state cache; a brief HA outage returns the last known list.

```
curl -s -G "$BASE/automations" -H "Authorization: Bearer $TOKEN" \
  --data-urlencode "user_ref=$USER" --data-urlencode "home_id=$HOME"
```

```
200 (verified on scott_home, 54 automations – abridged):
{ "home_id": "scott_home", "count": 54, "gate_check_available": true,
  "items": [ {
    "entity_id":      "automation.lights_off_at_night",
    "automation_id":  "lights_off_at_night",
    "ha_automation_id": "1751405011005",
    "name":           "Lights Off at Night",
    "enabled":         true, "state": "on",
    "last_triggered":  "2026-08-20T04:00:00+00:00",
    "mode": "single", "is_running": false,
    "icon": "mdi:robot",
    "voice_gated": false, "voice_auth_enrollment_id": null,
    "is_favorited": false, "favorite_id": null }, ... ] }
```

Param	Req	Notes
user_ref	yes	resolves the per-user bits: voice_gated + is_favorited (the automation list itself is home-scoped)
home_id	yes	which home's HA to read
q	no	case-insensitive substring on name + entity_id
enabled	no	true → only enabled; false → only disabled
limit	no	default 200, max 500

Field	Meaning
automation_id	entity suffix — the id the voice-gate / enrollment APIs key on
ha_automation_id	HA's own config id (editor/trace URLs). Null for YAML-defined automations
enabled	state == "on". Toggle it with action turn_on / turn_off (below)
last_triggered / mode / is_running	straight from HA; is_running = an instance is executing right now
voice_gated	true → running it answers 409; voice_auth_enrollment_id names the enrollment
gate_check_available	top-level; false = enrollment lookup failed, voice_gated flags defaulted to false and a run may still 409

Errors: 400 VALIDATION (missing params) · 404 NOT_CONFIGURED (unknown home) · 503 HOME_UNREACHABLE (HA down, nothing cached).

POST /automations/trigger — run / toggle / turn on / turn off ANY domain

The name is historical — this endpoint controls every entity domain, not just automations. It fires POST /api/services/{ha_service}/{action} on the home's HA. action is optional — omitted, the server picks the per-domain default. If an ACTIVE enrollment exists for (user_ref, automation_id) the call refuses with 409 ENROLLMENT_REQUIRED regardless of action — route to VAPI. user_ref + automation_id are required for that gate check; omitting them bypasses it (only for clearly unprotected items).

```
# run an automation now (automation domain default = trigger):
curl -s -X POST "$BASE/automations/trigger" -H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" -d '{
  "home_id": "scott_home", "ha_service": "automation",
  "ha_entity": "lights_off_at_night",
  "user_ref": "scott_mobile", "automation_id": "lights_off_at_night" }'
```

```
# toggle a light – what a boards-page tile tap sends:
-d '{ "home_id": "scott_home", "ha_service": "light",
      "ha_entity": "man_land_lamp",
      "user_ref": "scott_mobile", "automation_id": "man_land_lamp",
      "action": "toggle" }'
```

```
# disable / re-enable an automation:
      "ha_service": "automation", "ha_entity": "lights_off_at_night",
      "action": "turn_off"    # or "turn_on"
```

```
200: {"success": true, "message": "ok", "status_code": 200, "latency_ms": 214}
```

Action matrix — every domain (verified against HA's service registry, Aug 20 2026)

Domain	Default action (no `action` sent)	toggle	turn_on / turn_off
light, switch, fan, cover, media_player, remote, input_boolean	turn_on	✓ (LIVE-verified: light, switch)	✓
script	turn_on (runs it)	✓ (toggle STOPS a running script)	✓
climate	turn_on	✓ — flips real HVAC power; avoid on casual tiles	✓
automation	trigger (RUNS it)	✓ but = enable/disable, NOT run	✓ = enable / disable
scene	turn_on	—	turn_on only (a scene can't be "off")
lock	unlock (always voice-gated via favorites)	—	— use action lock / unlock

The generic toggle call — one template, every on/off domain

ha_service is always the entity's own domain (the part before the dot in its entity_id) and ha_entity the part after it — split light.man_land_lamp and you have the whole call:

```
curl -s -X POST "$BASE/automations/trigger" \
-H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
-d '{
  "home_id":      "scott_home",
  "ha_service":   "light",          # entity domain: light|switch|fan|media_player|...
  "ha_entity":    "man_land_lamp",  # entity suffix (no dot)
  "user_ref":     "scott_mobile",
  "automation_id": "man_land_lamp", # same suffix — enables the 409 gate check
  "action":       "toggle"         # or turn_on / turn_off
}'

200: {"success": true, "message": "ok", "status_code": 200, "latency_ms": 173}
```

Do NOT send ha_service: homeassistant — the API refuses it with 400 (entity_id is assembled as ha_service.ha_entity, so HA's generic domain would target a nonexistent homeassistant.* entity, which HA 200-OKs and silently ignores). Live-verified end-to-end in production: light.man_land_lamp and switch.bat_sign, each on → toggle → off → toggle → on; the other domains are confirmed by the home HA's own /api/services registry (climate, media_player, remote were deliberately not live-fired — real HVAC and TVs).

Why toggle exists. The default for a light is turn_on, so a bare trigger call can never turn anything OFF — that was the boards-page “Man Land Lamp won't turn off” bug. On/off tiles must send action: toggle and let HA resolve the flip from its own live state (immune to a stale UI). When checking the result, remember reads go through a ~10s state cache — wait ~12s before re-reading.

Copy-paste test — no variables, run exactly as printed (verified Aug 20, 2026)

Everything inline (production URL + the iOS platform key), so it runs from any terminal as-is. JSON has no comments — don't paste annotated bodies. Toggles Man Land Lamp (Downstairs Bat Cave) off and back on:

```
# 1. flip the lamp (toggle = whatever it is now, becomes the opposite):
curl -s -X POST "https://voiceorchestrator.homeadapt.us/api/v1/voice-auth/automations/trigger" \
-H "Authorization: Bearer sk_ios_48b546d143dae04ec9d2c4396ae9155648e80a5ffbf3377" \
-H "Content-Type: application/json" \
-d '{"home_id":"scott_home","ha_service":"light","ha_entity":"man_land_lamp",
  "user_ref":"scott_mobile","automation_id":"man_land_lamp","action":"toggle"}'
# -> {"latency_ms":474,"message":"ok","status_code":200,"success":true}

# 2. confirm the state — WAIT ~12s first (reads come from a 10s cache):
sleep 12; curl -s -G "https://voiceorchestrator.homeadapt.us/api/v1/voice-auth/items/search" \
-H "Authorization: Bearer sk_ios_48b546d143dae04ec9d2c4396ae9155648e80a5ffbf3377" \
--data-urlencode "user_ref=scott_mobile" --data-urlencode "home_id=scott_home" \
--data-urlencode "q=man_land_lamp"
# -> ... "name":"Man Land Lamp","state":"off" ...

# 3. run step 1 again to flip it back on (leave it as you found it).
# Guaranteed direction instead of a flip: "action":"turn_off" or "turn_on"
# (idempotent - safe to repeat when you don't know the current state).
```

Test recipe — prove toggle/on/off on any domain from a terminal

```
fire() { # fire <domain> <entity-suffix> <action>
  curl -s -X POST "$BASE/automations/trigger" \
    -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
    -d '{"home_id\":"$HOME_ID\","ha_service\":"$1\","ha_entity\":"$2\","user_ref\":"$USER_REF\","automation_id\":"$2\","action\":"$3\"}"; }

state() { # current state of anything matching $1
  curl -s -H "Authorization: Bearer $KEY" \
    "$BASE/items/search?user_ref=$USER_REF&home_id=$HOME_ID&q=$1"; }

state man_land_lamp          # note the state
fire light man_land_lamp toggle # {"success": true, ...}
sleep 12; state man_land_lamp # state flipped
fire light man_land_lamp toggle # flip it back

fire switch bat_sign turn_off # explicit off
fire switch bat_sign turn_on  # explicit on
fire automation lights_off_at_night turn_off # DISABLE the automation
fire automation lights_off_at_night turn_on  # re-enable it
```

7 · Boards (HA dashboards as tap-to-control tiles)

Summarized here for completeness — the full contract (entity_meta, icon resolution, strategy boards, run book) is in [boards_api_guide.pdf](#) v1.3.

```
# the home's dashboards (default "Overview" always first, url_path null):
curl -s -G "$BASE/dashboards" -H "Authorization: Bearer $TOKEN" \
  --data-urlencode "user_ref=$USER" --data-urlencode "home_id=$HOME"

# one board's views + entities + per-entity tile metadata:
curl -s -G "$BASE/dashboards/config" -H "Authorization: Bearer $TOKEN" \
  --data-urlencode "user_ref=$USER" --data-urlencode "home_id=$HOME" \
  --data-urlencode "url_path=tablet" # omit for Overview
# tile UIs add: include_categories=primary&include_hidden=false
```

- Each entity in entity_meta carries name, real HA icon, category, live state, controllable — no deriving names from entity IDs.
- Voice-gated entities are withheld from boards by default (listed in gated_excluded), so every returned tile is a plain button click: on/off domains send action toggle (§6), scenes/scripts a bare trigger call.
- 409 DASHBOARD_NOT_CONFIGURED = the board was never saved in HA (open it there, edit, save once)
- 404 DASHBOARD_NOT_FOUND = unknown url_path.

8 · Voice enable (provision a VAPI phone number)

```
curl -s -X POST "$BASE/voice-enable" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"user_ref":"scott_mobile","home_id":"scott_home","area_code":"415"}'
```

201 {phone_e164, vapi_phone_number_id, is_active}. Idempotent per (user_ref, home_id) — no double purchase; first call is billable on VAPI. GET /voice-enable?user_ref=... → {enabled, is_dry_run, mapping}. DELETE /voice-enable?user_ref=... releases the number → 204.

Phone mappings (server/VAPI internal): POST/GET /phone-mappings, DELETE /phone-mappings/{id}, GET /phone-lookup — used by the inbound VAPI webhook to resolve caller → (user_ref, home_id). The app normally doesn't call these.

9 · VAPI SDK — the spoken challenge

On 409 ENROLLMENT_REQUIRED (from trigger or favorite-fire), launch the VAPI SDK (iOS: github.com/VapiAI/ios, Android: /android, Web: /web) with the **VAPI public key** (not the API key):

```
assistantId: "1a2904b1-61cf-49da-a804-199d8d39fb9f"
assistantOverrides.variableValues:
  home_id:      "scott_home"
  user_ref:     "scott_mobile"
  automation_id: "main_lights_on"
  automation_name: "Main Lights On"
  initiated_by:  "MOBILE_IOS" | "MOBILE_ANDROID"
```

Assistant prompts the security phrase; on pass, HA fires. Subscribe to the call-end event, then refresh with GET /check.

10 · Error reference (all endpoints)

HTTP	Code	Meaning	App action
400	VALIDATION	bad body/params	fix input, inline error
400	NO_CONTROLLABLE_ENTITY	sensor-only device	exclude from picker
401	UNAUTHORIZED	missing/expired token or bad key	re-login (token) / config bug (key)
403	FORBIDDEN	foreign user_ref or home_id with a token	app bug — use own identity
403	PENDING_APPROVAL	account awaiting activation	show “awaiting activation”
404	NOT_FOUND	unknown id/ref	varies; delete-by-ref hint included
404	DASHBOARD_NOT_FOUND	unknown board url_path	refresh the boards list
404	NOT_CONFIGURED	unknown home_id	check home_id from login payload
409	ENROLLMENT_REQUIRED	voice-gated (always locks)	launch VAPI session
409	EMAIL_EXISTS	signup email taken	offer login
409	DASHBOARD_NOT_CONFIGURED	board never saved in HA	tell user to save it once in HA
429	RATE_LIMITED	login/signup lockout (15 min)	“try later”, no auto-retry
502	— / VAPI_ERROR	HA or VAPI upstream failure	retry once with backoff
503	HOME_UNREACHABLE	home's HA down or token lapsed	“home offline, retry later”
503	NOT_CONFIGURED	feature not wired	report to backend

Support: send user_ref, the HTTP request+response (curl -i), and a vapi_call_id from GET /challenges for voice flows. Companion docs: boards_api_guide.pdf (boards deep dive + run book) · identity_api_guide.pdf (login/sign-up deep dive) · mobile_handoff.md (narrative walkthroughs).